

ICT600 TEACHING EDITION

Corrected Teaching Answer Scheme

July 2026 final examination | 100 marks | evidence-aligned and code-checked

Scope	July 2026 final examination
Evidence	Official question + official answer + generated scheme + current lecture notes
Status	Pilot complete; originals remain unchanged

Prepared 26 September 2026

How to use this scheme

Status

This is a corrected teaching edition, not a replacement official document. It preserves the printed question marks, documents acceptable alternatives, and fixes technical or safety defects found during audit.

- Award marks for equivalent technically correct wording.
- Where the question stem is weaker than the mark demand, apply the explicit marking guide below consistently.
- For scenario questions, reward a valid identification only when the explanation fits the scenario where the rubric requires assessment.
- For coding questions, accept equivalent working solutions that meet the stated behavior and validation requirements.
- Part A: 70 marks. Part B: 30 marks. Total: 100 marks.

Part A - Short and structured responses (70 marks)

Question 1(a)

Define URL, HTTP, HTTPS, and Web Application.

[8 marks]

- URL: Uniform Resource Locator; the address used to identify and locate a resource on the Web.
- HTTP: Hypertext Transfer Protocol; the request-response protocol used for communication between web clients and servers.
- HTTPS: HTTP carried over a TLS-encrypted connection to protect data in transit and authenticate the server.
- Web application: software accessed through a web browser and delivered over a network; processing may occur on the client, the server, or both.

Marking guide: 2 marks each: correct term/expansion or identity (1) + accurate function or distinguishing detail (1).

Question 1(b)

Identify FIVE categories of Web Applications.

[10 marks]

- Static web application - delivers largely fixed content; changing content usually requires editing and redeploying files.
- Dynamic web application - generates or updates content using application logic and commonly a database/CMS.
- Online shop/e-commerce application - supports browsing, selection, ordering, and payment for products or services.
- Portal web application - brings multiple services or content areas together through a home page, such as email, forums, search, and news.
- Animated web application - emphasizes animated or highly interactive presentation effects.
- Web application with CMS - uses a content-management system such as WordPress, Joomla, or Drupal so authorised users can manage content.

Marking guide: Any five: category name (1) + suitable brief explanation (1) = 10.

Teaching note

The printed stem says identify, but ten marks and the official intent require a brief explanation. The official portal explanation was duplicated from animated web application and is corrected here.

Question 2(a)

Define JavaScript, HTML Element, and HTML Attribute.

[6 marks]

- JavaScript: a scripting language commonly used to add dynamic behaviour and interactivity to web pages, traditionally executed in the browser.
- HTML element: the complete HTML component represented by a start tag, content, and usually an end tag; some elements are void elements.
- HTML attribute: additional information or configuration written in an element's start tag, normally as a name/value pair.

Marking guide: 2 marks per definition: accurate core meaning (1) + distinguishing detail/example (1).

Question 2(b)

State THREE HTML formatting elements.

[6 marks]

- `<b>` - bold text without added semantic importance.
- `<strong>` - text of strong importance, normally rendered bold.
- `<i>` - text in an alternate voice or convention, normally rendered italic.
- `<em>` - emphasized text, normally rendered italic.
- Also accept `<mark>`, `<small>`, `<del>`, `<ins>`, `<sub>`, or `<sup>` with the correct effect.

Marking guide: Any three: valid element/tag (1) + correct effect/meaning (1) = 6.

Teaching note

The word state alone normally suggests one mark per item; the added explanation makes the six-mark intention explicit.

Question 3(a)

Prepare an SQL statement to create library_books with book_id, title, author, and year_published.

[8 marks]

- `CREATE TABLE library_books (`
- `book_id INT AUTO_INCREMENT PRIMARY KEY,`
- `title VARCHAR(255) NOT NULL,`
- `author VARCHAR(255) NOT NULL,`
- `year_published INT`
- `);`

Marking guide: `CREATE TABLE` and table name (2); opening parenthesis (0.5); `book_id` line including `AUTO_INCREMENT PRIMARY KEY` (2); `title` (1); `author` (1); `year_published` (1); closing parenthesis/semicolon (0.5).

Teaching note

`AUTO_INCREMENT` is retained because the official rubric explicitly awards marks for it, although the printed question does not expressly require automatic numbering.

Question 3(b)

Prepare the five required SELECT statements for the products table.

[10 marks]

- Use the exact column names and conditions printed in each subpart.
- Award 2 marks per query: correct `SELECT`/column expression (1) + correct `FROM` and required `WHERE`/`ORDER` condition (1).
- Equivalent valid MySQL syntax receives full credit.

Marking guide: Five queries x 2 marks = 10.

Question 4(a)

Discover FOUR popular web application frameworks.

[4 marks]

- Any four valid web application frameworks, for example: Laravel, CodeIgniter, CakePHP, Django, Ruby on Rails, Spring, ASP.NET Core MVC, or Express.

Marking guide: 1 mark per valid framework, maximum 4.

Teaching note

For future papers, replace discover with list or state. Do not award a mark for bare Node.js: it is a JavaScript runtime. Express is a web framework that runs on Node.js.

Question 4(b)

Explain THREE framework challenges for a small corporate-profile project.

[6 marks]

- Learning curve - the team must learn framework conventions even though the project is small.
- Common code may not be optimized - framework overhead can be unnecessary for a simple site.
- Framework security/dependency risk - flaws or outdated dependencies can affect the application.
- Reduced flexibility - conventions may constrain a custom or lightweight implementation.

Marking guide: Any three: valid challenge (1) + project-relevant explanation (1) = 6.

Question 5(a)

Classify THREE common web vulnerabilities relevant to malicious input in a login scenario.

[6 marks]

- SQL injection - crafted input changes the meaning of a database query and may bypass login controls.
- Cross-site scripting (XSS) - untrusted input is returned to a page and executed as script in another user's browser.
- Broken authentication/session management - weaknesses in credentials, session IDs, or login controls let an attacker impersonate a user.
- Accept other course vulnerabilities when correctly described and credibly connected to the scenario.

Marking guide: Any three: vulnerability (1) + accurate scenario-relevant description (1) = 6.

Teaching note

The printed verb classify is not reflected by the original answer. For future use, ask students to identify and explain, or supply categories to classify into.

Question 5(b)

Assess THREE malware types relevant to a suspicious attachment, unusual computer behaviour, and unwanted advertisements.

[6 marks]

- Trojan horse - a malicious program disguised as a legitimate attachment; it may remain hidden and perform unauthorised actions.
- Adware - software that displays or downloads unwanted advertisements and may collect marketing data.
- Spyware - covertly gathers and transmits information without the user's knowledge.
- Virus - malicious code that infects files or systems and may cause unusual behaviour; accept when the response links it to observed symptoms.
- Other malware types may be accepted if the assessment matches the scenario evidence.

Marking guide: Any three: malware type (1) + correct scenario-linked assessment (1) = 6.

Part B - Application and coding (30 marks)

Question 1 - BMI calculator

Develop an HTML and JavaScript BMI calculator that accepts weight in kilograms and height in metres, validates input, calculates BMI, determines a category, and displays the result.

[15 marks]

Criterion	Marks
HTML document structure	2
Weight, height, button, result elements	3
JavaScript function and event	2
Numeric and positive-value validation	3
BMI formula	2
Category selection	2
Display result	1
Total	15

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>BMI Calculator</title>
</head>
<body>
  <h1>BMI Calculator</h1>
  <label>Weight (kg): <input id="weight" type="number" min="1" step="0.1"></label>
  <label>Height (m): <input id="height" type="number" min="0.1" step="0.01"></label>
  <button type="button" onclick="calculateBMI()">Calculate</button>
  <p id="result" aria-live="polite"></p>

  <script>
    function calculateBMI() {
      const weight = Number(document.getElementById('weight').value);
      const height = Number(document.getElementById('height').value);

      const result = document.getElementById('result');

      if (!Number.isFinite(weight) || !Number.isFinite(height) ||
        weight <= 0 || height <= 0) {
        result.textContent = 'Enter positive numeric values.';
        return;
      }

      const bmi = weight / (height * height);
      let category;
      if (bmi < 18.5) category = 'Underweight';
      else if (bmi < 25) category = 'Normal';
      else if (bmi < 30) category = 'Overweight';
      else category = 'Obese';

      result.textContent = `BMI: ${bmi.toFixed(2)} (${category})`;
    }
  </script>
</body>
</html>
```

Equivalent solutions

Accept addEventListener instead of onclick, alternative valid form markup, and equivalent thresholds if a defensible category standard is stated. Award behavior and correctness, not exact formatting.

Question 2 - Student registration form

Construct an HTML/PHP Student Registration Form with name, student ID, gender, and programme; submit using POST; validate empty values; and show the submitted details or an unsuccessful message.

[15 marks]

Criterion	Marks
Form structure	3
Name, student ID, programme inputs	3
Gender radio controls	2
POST method and request handling	2
empty() validation	3
Success/error output	2
Total	15

```
<?php
$name = $studentId = $gender = $programme = '';
$message = '';
$submitted = false;

function e(string $value): string {
    return htmlspecialchars($value, ENT_QUOTES, 'UTF-8');
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = trim($_POST['name'] ?? '');
    $studentId = trim($_POST['student_id'] ?? '');
    $gender = trim($_POST['gender'] ?? '');
    $programme = trim($_POST['programme'] ?? '');

    if (empty($name) || empty($studentId) ||
        empty($gender) || empty($programme)) {
        $message = 'Unsuccessful: all fields are required.';
    } else {
        $submitted = true;
        $message = 'Registration successful.';
    }
}
?>
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Student Registration</title>
</head>
<body>
    <h1>Student Registration Form</h1>
    <form method="post" action="">
        <label>Name: <input name="name" value="<? e($name) ?>" required></label><br>
        <label>Student ID: <input name="student_id" value="<? e($studentId) ?>" required></label><br>
        <span>Gender:</span>
        <label><input type="radio" name="gender" value="Male" required
            <? $gender === 'Male' ? 'checked' : '' ?>> Male</label>
        <label><input type="radio" name="gender" value="Female"
            <? $gender === 'Female' ? 'checked' : '' ?>> Female</label><br>
        <label>Programme: <input name="programme" value="<? e($programme) ?>" required></label><br>
        <button type="submit">Register</button>
    </form>

    <?php if ($message !== ''): ?>
        <p><? e($message) ?></p>
    <?php endif; ?>

    <?php if ($submitted): ?>
        <h2>Submitted Details</h2>
        <p>Name: <? e($name) ?></p>
        <p>Student ID: <? e($studentId) ?></p>
        <p>Gender: <? e($gender) ?></p>
        <p>Programme: <? e($programme) ?></p>
    <?php endif; ?>
</body>
</html>
```

Required safety property

Submitted values are encoded with htmlspecialchars before being inserted into HTML. This prevents user-controlled markup or script from being interpreted by the browser.

Marker consistency notes

- Do not double-penalise a single syntax error that propagates through an otherwise coherent answer.
- Award method marks when the student's approach is correct and the remaining error is minor.
- For code questions, preserve the official category totals even when accepting an equivalent implementation.
- When a response gives more answers than requested, apply the institution's established excess-answer policy consistently.
- Record accepted alternatives discovered during marking and add them to the next controlled scheme revision.

Traceability of corrections

Correction	Reason/evidence
Portal definition replaced	Chapter 1 describes portal access through a home page with sections such as chats, forums, and email.
Framework mapped to Chapter 7	Current lecture file is 7 - Framework.pdf; official scheme says Topic 8.
Bare Node.js excluded	Framework examples in current notes include CakePHP, CodeIgniter, Laravel, and Django.
Q5 alternatives broadened	Chapter 9 presents multiple vulnerabilities and distinct malware types; scenario evidence should control credit.
PHP output encoded	Chapter 9 page 100 requires input validation and output encoding.
Official Part B rubrics restored	Generated allocations differed from the official category allocations despite equal totals.